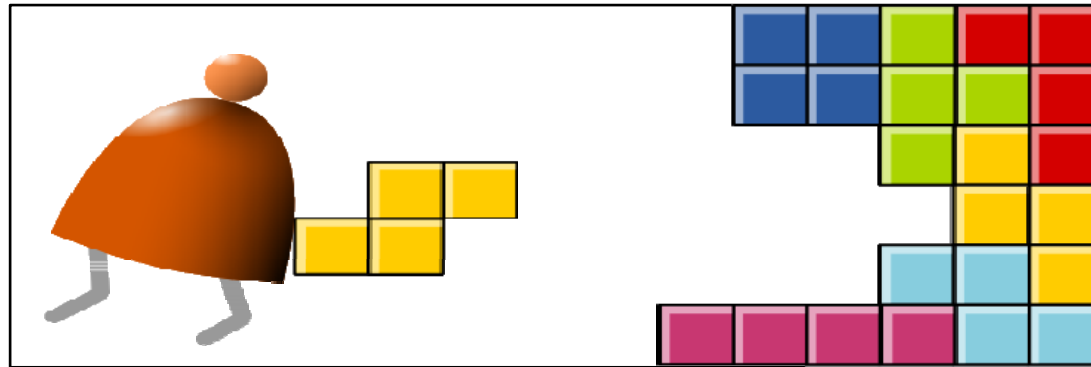


Boolean Logic



Building a Modern Computer From First Principles

www.nand2tetris.org



Chapter 1: Boolean logic

Theory



Basic concepts

- Nand

Practice

- Logic gates
- HDL
- Hardware simulation
- Multi-bit buses

Project 1

- Introduction
- Chips
- Guidelines



Boolean values



Boolean values



Boolean values

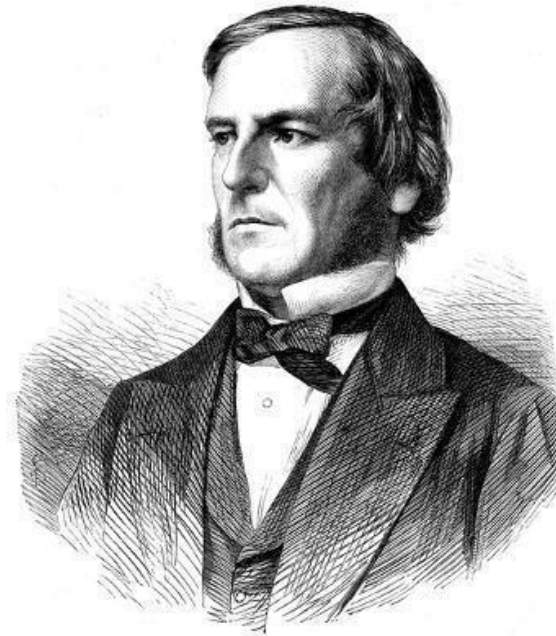


off no false 0



on yes true 1

Different labels, all referring to two possible states.



George Boole

1815 - 1864



Boolean values

b_1 b_0



1 binary variable: 2 possible states



Boolean values

b_1 b_0



1 binary variable: 2 possible states

2 binary variables: 4 possible states



Boolean values

b_2 b_1 b_0



1 binary variable: 2 possible states

2 binary variables: 4 possible states



Boolean values

... b_2 b_1 b_0



1 binary variable: 2 possible states

2 binary variables: 4 possible states

3 binary variables: 8 possible states

...



Truth Table

- A truth table lists all possible input combinations and the resulting output.
- Used to understand how logical expressions behave (Boolean logic).
- Columns = variables and expressions; rows = all input possibilities.

a	b
0	0
0	1
1	0
1	1



How Many Rows in a Truth Table?

□ If there are n input variables, the truth table has 2^n rows.

- 1 input $\rightarrow 2^1 = 2$ rows
- 2 inputs $\rightarrow 2^2 = 4$ rows
- 3 inputs $\rightarrow 2^3 = 8$ rows

□ This ensures every possible combination is covered.

2^3

a	b	c
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

2^2

a	b
0	0
0	1
1	0
1	1

2^1

a
0
1



Boolean functions

x	y	f
		
		
		
		



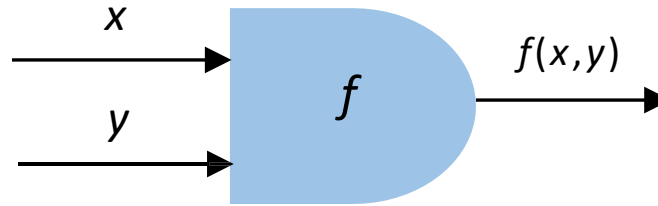
Boolean functions

x	y	f
0	0	0
0	1	0
1	0	0
1	1	1



Boolean functions

x	y	f
0	0	0
0	1	0
1	0	0
1	1	1

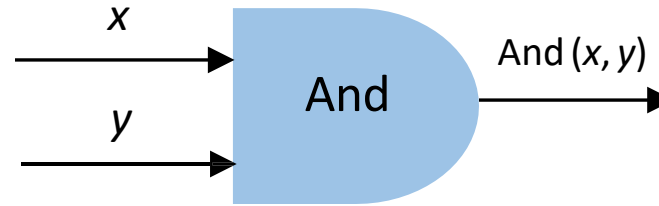


$$f(x,y) = \begin{cases} 1 & \text{when } x == 1 \text{ and } y == 1 \\ 0 & \text{otherwise} \end{cases}$$



Boolean functions

x	y	And
0	0	0
0	1	0
1	0	0
1	1	1



$$\text{And}(x,y) = \begin{cases} 1 & \text{when } x == 1 \text{ and } y == 1 \\ 0 & \text{otherwise} \end{cases}$$

Boolean function

A function that operates on boolean variables, and returns a boolean value

Simple boolean functions (like **And**):

- Sometimes called *operators*
- Their variables are called *operands*
- $f(x, y)$ can also be written as $x \text{ } f \text{ } y$



Elementary Boolean functions

- $\text{Not}(x)$
- $\text{And}(x,y)$
- $\text{Or}(x,y)$
- $\text{Nand}(x,y)$



AND ($\wedge$) — Both Must Be True

- Example:** You can log in only if you enter the correct username and the correct password.

Correct Username (A)	Correct Password (B)	Can Log In (A $\wedge$ B)
No (0)	No (0)	No (0)
No (0)	Yes (1)	No (0)
Yes (1)	No (0)	No (0)
Yes (1)	Yes (1)	Yes (1)



OR ($\vee$) — At Least One is True

Example: You see a warning if your laptop battery is low or your disk is almost full.

Low Battery (A)	Disk Almost Full (B)	Warning (A $\vee$ B)
No (0)	No (0)	No (0)
No (0)	Yes (1)	Yes (1)
Yes (1)	No (0)	Yes (1)
Yes (1)	Yes (1)	Yes (1)



NOT ($\neg$) — Opposite of input

Example: If Wi-Fi is NOT connected, show 'No Internet'.

Wi-Fi Connected (A)	Show "No Internet" ($\neg A$)
No (0)	Yes (1)
Yes (1)	No (0)



Boolean algebra (Recap)

a	Not (a)
0	1
1	0

a	b	AND (a, b)
0	0	0
0	1	0
1	0	0
1	1	1

a	b	Xor(a,b)
0	0	0
0	1	1
1	0	1
1	1	0

x	y	z	$f(x, y, z) = (x + y)\bar{z}$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

a	b	OR (a, b)
0	0	0
0	1	1
1	0	1
1	1	1

a	b	NAND (a, b)
0	0	1
0	1	1
1	0	1
1	1	0

- A Boolean function can be expressed using a functional expression or a truth table expression
- Important observation:
Every Boolean function can be expressed using And, Or, Not.



Boolean functions

x And y

x	y	And
0	0	0
0	1	0
1	0	0
1	1	1

x Or y

x	y	Or
0	0	0
0	1	1
1	0	1
1	1	1

Not(x)

y	Not
0	1
1	0

Boolean function evaluation (example):



Boolean functions

x And y

x	y	And
0	0	0
0	1	0
1	0	0
1	1	1

x Or y

x	y	Or
0	0	0
0	1	1
1	0	1
1	1	1

Not(x)

y	Not
0	1
1	0

Boolean function evaluation (example):

Not(x Or (y And z))

Evaluate this function for, say,
x = 0, y = 1, z = 1

Not(0 Or (1 And 1)) =

Not(0 Or 1) =

Not(1) =

0

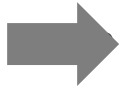


Chapter 1: Boolean logic

Theory



Basic concepts



Nand

Practice

- Logic gates
- HDL
- Hardware simulation
- Multi-bit buses

Project 1

- Introduction
- Chips
- Guidelines



The expressive power of Nand

$x \text{ Nand } y$			$x \text{ And } y$			$x \text{ Or } y$			$\text{Not}(x)$	
x	y	Nand	x	y	And	x	y	Or	x	Not
0	0	1	0	0	0	0	0	0	0	1
0	1	1	0	1	0	0	1	1	1	0
1	0	1	1	0	0	1	0	1	0	1
1	1	0	1	1	1	1	1	1	1	0

Observations

- **Not**(x) = x **Nand** x
- x **And** y = **Not**(x **Nand** y)
- x **Or** y = **Not**(**Not**(x) **And** **Not**(y))
(De Morgan)

Thus:

- **Not** can be realized using **Nand**
- **And** can be realized using **Nand**
- **Or** can be realized using **Nand**

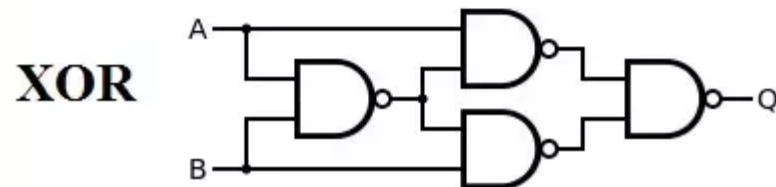
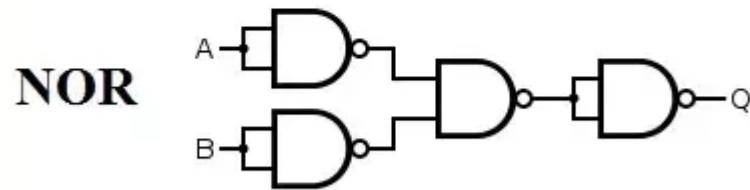
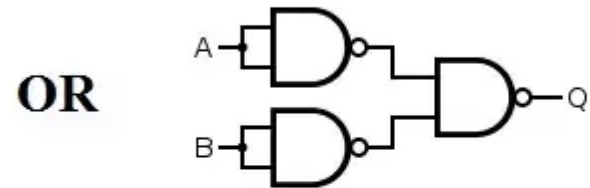
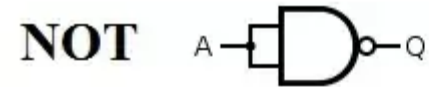
Theorem: Any Boolean function can be realized using only Nand.

Proof : Any Boolean function can be expressed as a truth table. Any truth table can be expressed as a Boolean function using only Not, And, and Or (synthesized as a DNF, Disjunctive Normal Form). Combined with the above observations, Q.E.D.



The expressive power of Nand

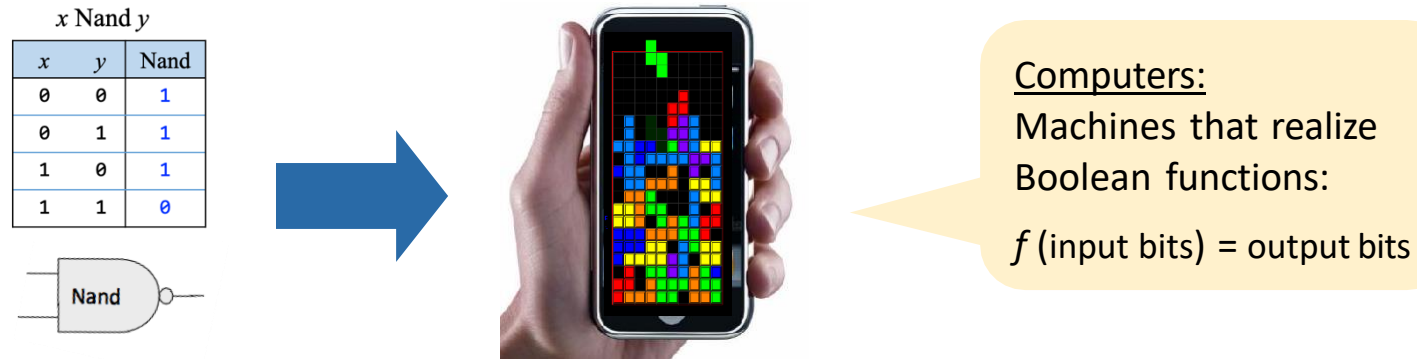
Theorem: Any Boolean function can be realized using only Nand.



The expressive power of Nand

Theorem: Any Boolean function can be realized using only Nand.

Implication: Any computer can be built from Nand gates only:



OK, so we *can* build a computer from Nand gates only.

But... how can we *actually* do it?

That's what the Nand to Tetris course is all about!

NAND ($\neg(A \wedge B)$) — Opposite of AND

Example: Your computer only stays ON if both power cable and battery are working. If either one fails, it turns OFF (NAND logic showing failure).

Power Cable OK (A)	Battery OK (B)	$A \wedge B$	System Fails ($\neg(A \wedge B)$)
No (0)	No (0)	0	Yes (1)
No (0)	Yes (1)	0	Yes (1)
Yes (1)	No (0)	0	Yes (1)
Yes (1)	Yes (1)	1	No (0)



All Boolean functions of 2 variables

Function	x	0	0	1	1
	y	0	1	0	1
Constant 0	0	0	0	0	0
And	$x \cdot y$	0	0	0	1
x And Not y	$x \cdot \bar{y}$	0	0	1	0
x	x	0	0	1	1
Not x And y	$\bar{x} \cdot y$	0	1	0	0
y	y	0	1	0	1
Xor	$x \cdot \bar{y} + \bar{x} \cdot y$	0	1	1	0
Or	$x + y$	0	1	1	1
Nor	$\overline{x + y}$	1	0	0	0
Equivalence	$x \cdot y + \bar{x} \cdot \bar{y}$	1	0	0	1
Not y	$\bar{y}$	1	0	1	0
If y then x	$x + \bar{y}$	1	0	1	1
Not x	$\bar{x}$	1	1	0	0
If x then y	$\bar{x} + y$	1	1	0	1
Nand	$\overline{x \cdot y}$	1	1	1	0
Constant 1	1	1	1	1	1

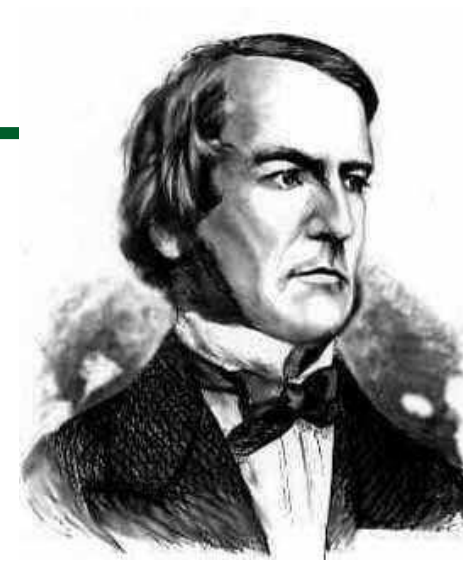


Boolean algebra

Given: $\text{Nand}(a,b), \text{false}$

We can build:

- $\text{Not}(a) = \text{Nand}(a,a)$
- $\text{true} = \text{Not}(\text{false})$
- $\text{And}(a,b) = \text{Not}(\text{Nand}(a,b))$
- $\text{Or}(a,b) = \text{Not}(\text{And}(\text{Not}(a), \text{Not}(b)))$
- $\text{Xor}(a,b) = \text{Or}(\text{And}(a, \text{Not}(b)), \text{And}(\text{Not}(a), b))$
- Etc.



George Boole, 1815-1864
(*"A Calculus of Logic"*)



Logic gates

➡ Elementary gates (Nand, And, Or, Not, ...)

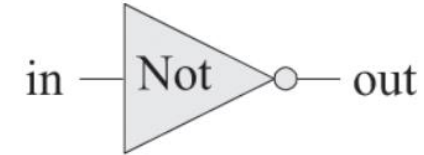
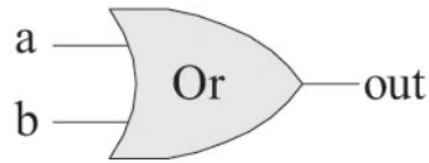
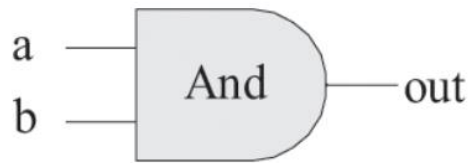
- Composite gates (Mux, Adder, ...)



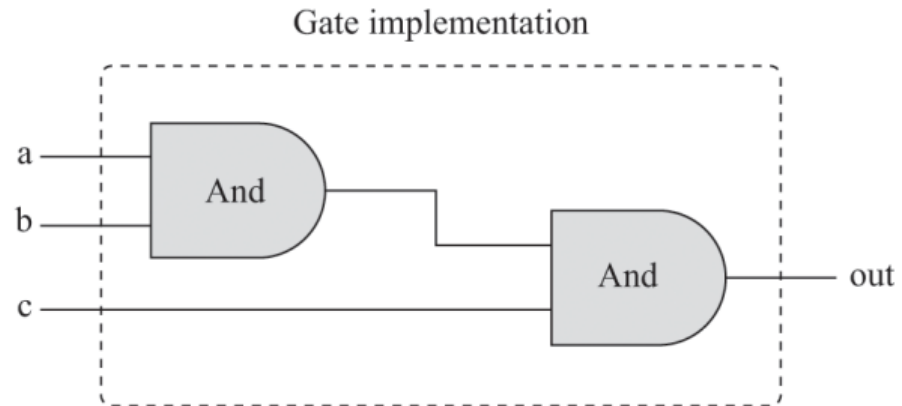
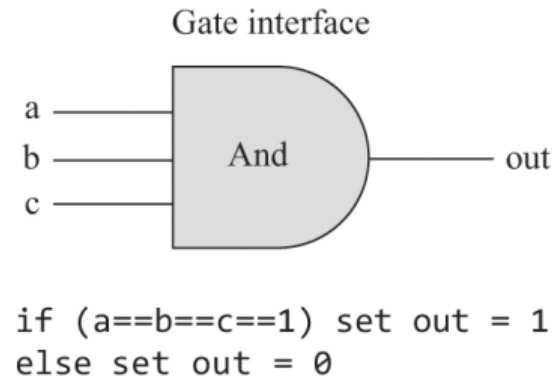
Gate logic

- Gate logic – a gate architecture designed to implement a Boolean function

- Elementary gates:



- Composite gates:



- Important distinction: Interface (*what*) VS implementation (*how*).



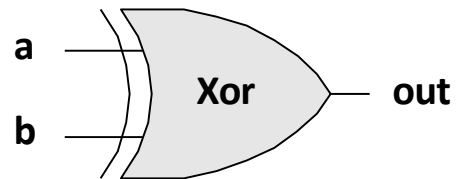
Gate logic



Claude Shannon, 1916-2001

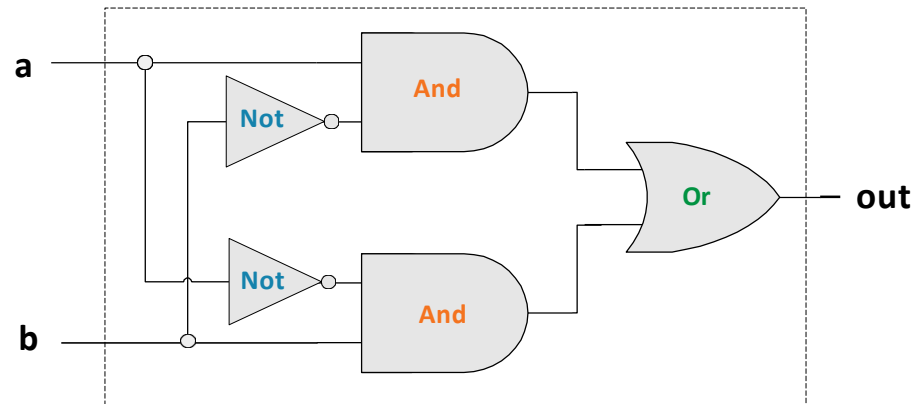
("Symbolic Analysis of Relay and Switching Circuits")

Interface



a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

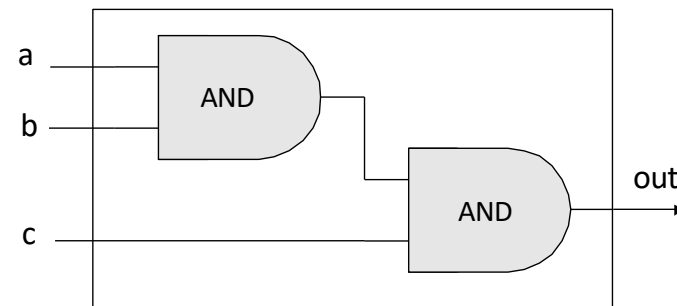
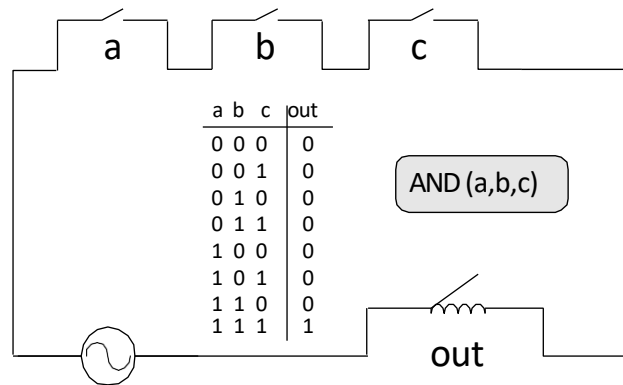
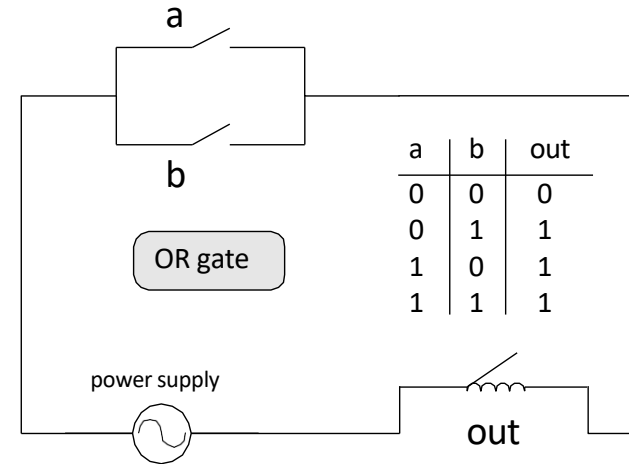
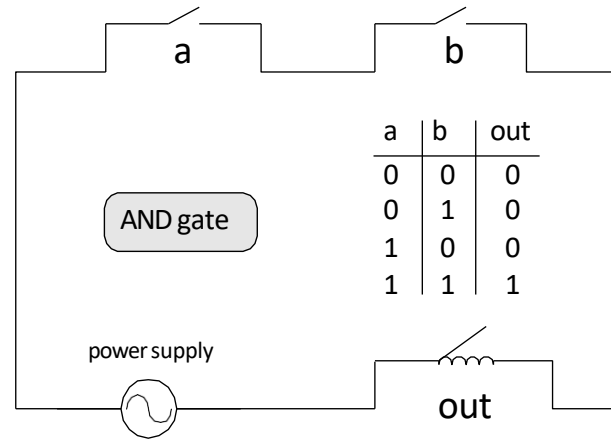
Implementation



$$\text{Xor}(a,b) = \text{Or}(\text{And}(a,\text{Not}(b)),\text{And}(\text{Not}(a),b))$$



Circuit implementations



- From a computer science perspective, physical realizations of logic gates are irrelevant.



Project 1

Given: Nand

Goal: Build the following gates:

Elementary logic gates

- Not
- And
- Or
- Xor
- Mux
- DMux

16-bit variants

- Not16
- And16
- Or16
- Mux16

Multi-way variants

- Or8Way
- Mux4Way16
- Mux8Way16
- DMux4Way
- DMux8Way

Why these 15 particular gates?

- Commonly used gates
- Comprise all the elementary logic gates needed to build our computer.



Multiplexer

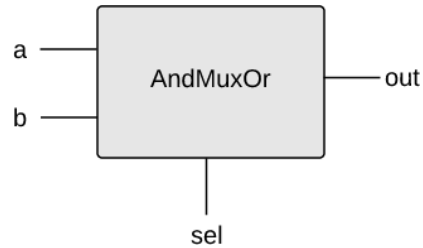
a	b	sel	out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

sel	out
0	a
1	b

Proposed Implementation: based on Not, And, Or gates.



Example 1: Using Mux logic to build a programmable gate

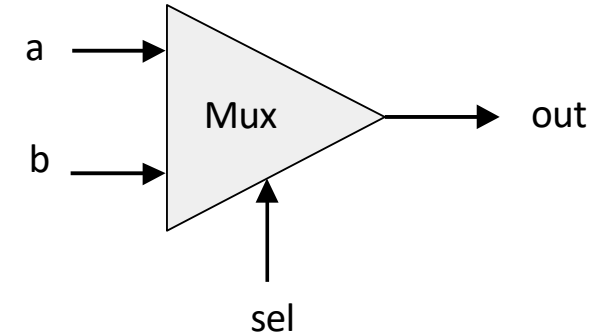


if (sel == 0)
out = a And b
else
out = a Or b

a	b	sel	out
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	1
1	1	1	1

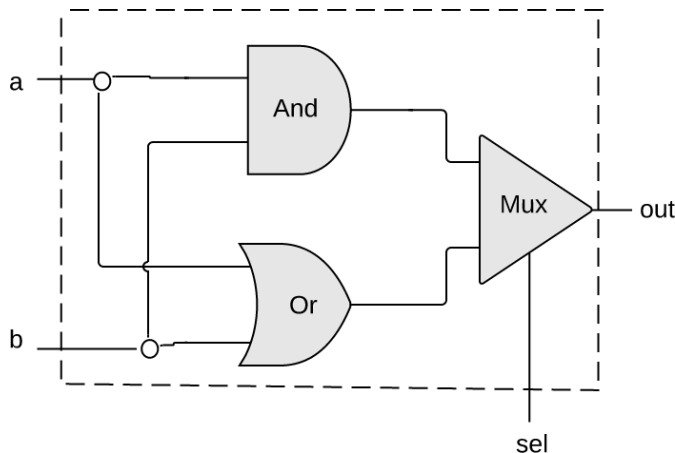
When sel == 0
the gate acts like an And gate

When sel == 1
the gate acts like an Or gate



sel	out
0	a
1	b

abbreviated
truth table

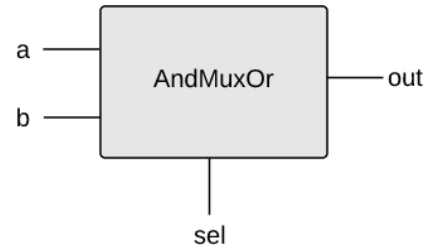


Mux.hdl

```
CHIP AndMuxOr {
  IN a, b, sel;
  OUT out;
```



Example 1: Using Mux logic to build a programmable gate

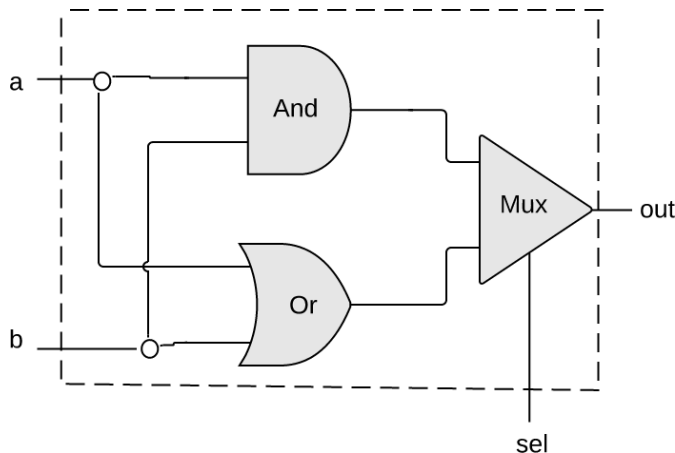


if (sel == 0)
out = a And b
else
out = a Or b

a	b	sel	out
0	0	0	0
0	1	0	0
1	0	0	0
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	1
1	1	1	1

When sel == 0
the gate acts like an And gate

When sel == 1
the gate acts like an Or gate



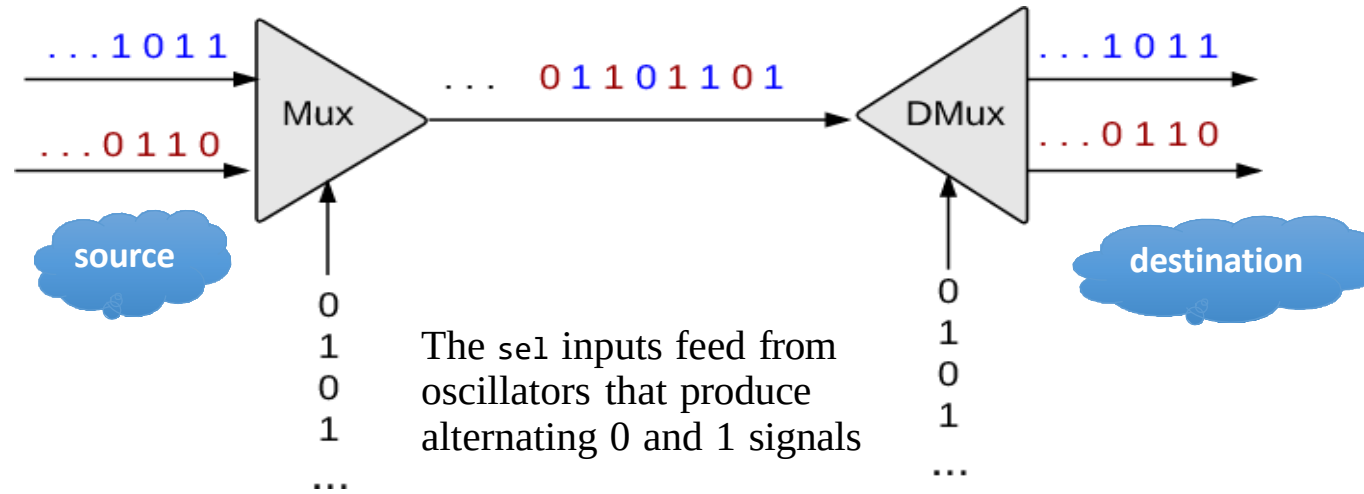
Mux.hdl

```
CHIP AndMuxOr {
  IN a, b, sel;
  OUT out;

  PARTS:
    And (a=a, b=b, out=andOut);
    Or (a=a, b=b, out=orOut);
    Mux (a=andOut, b=orOut, sel=sel, out=out);
}
```

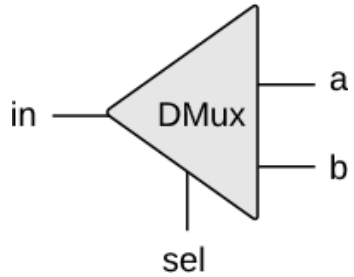


Example 2: Using Mux logic to build an interleaved channel



- Enables transmitting multiple messages simultaneously using a single, shared communications line
- *Conceptual, and unrelated to this course.*

Demultiplexer



if (sel == 0)
 $\{a, b\} =$
 $\{in, 0\}$ else
 $\{a, b\} = \{0, in\}$

in	sel	a	b
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

- The “inverse” of a multiplexor
- Routes the single input value to one of two possible destinations

DMux.hdl

```
CHIP DMux {
  IN in, sel;
  OUT a, b;

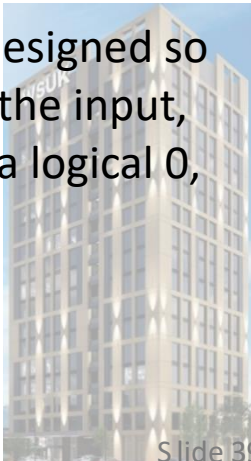
  PARTS:
    // Put your code here:
}
```

Implementation tip

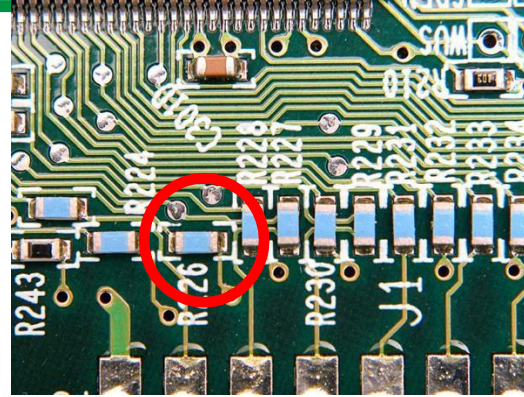
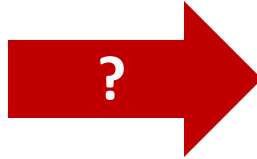
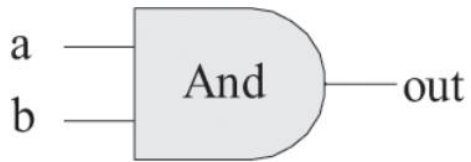
Simple truth table, simple implementation.

- When **S=0**, the demultiplexer is "selecting" output a.
 - If the input is IN=0, the output a is 0. All other outputs (just b here) are also 0.
 - If the input is IN=1, the output a is 1. The other output b is 0.
- When **S=1**, the demultiplexer is "selecting" output b.
 - If the input is IN=0, the output b is 0. The other output a is 0.
 - If the input is IN=1, the output b is 1. The other output a is 0.

The logic gates inside the demultiplexer are designed so that the selected output line directly mirrors the input, while all the non-selected lines are forced to a logical 0, regardless of what the input is.



Building a chip



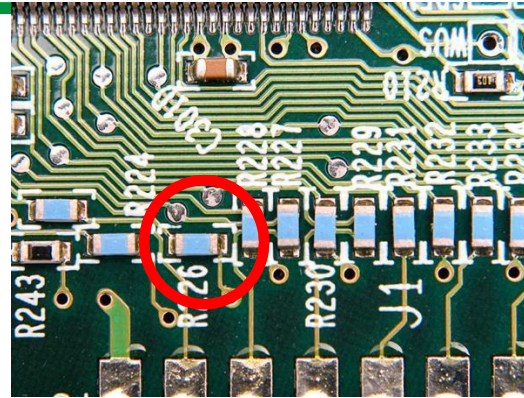
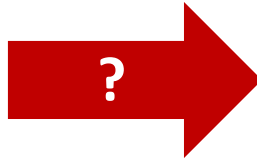
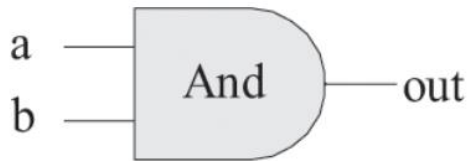
(an arbitrary image found on the Internet...)

The process

- Design the chip architecture
- Specify the architecture in HDL
- Test the chip in a hardware simulator
- Optimize the design
- Realize the optimized design in silicon.



Building a chip



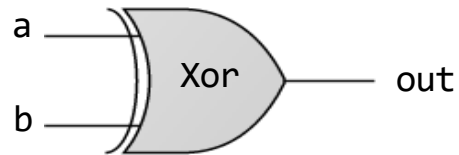
(an arbitrary image found on the Internet...)

The process

- ✓ Design the chip architecture
- ✓ Specify the architecture in HDL
- ✓ Test the chip in a hardware simulator
 - Optimize the design
 - Realize the optimized design in silicon.



Design: Requirements



```
if ((a == 0 and b == 1) or (a == 1 and b == 0))
    out = 1
else
    out = 0
```

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Requirement

Build a chip that delivers this functionality

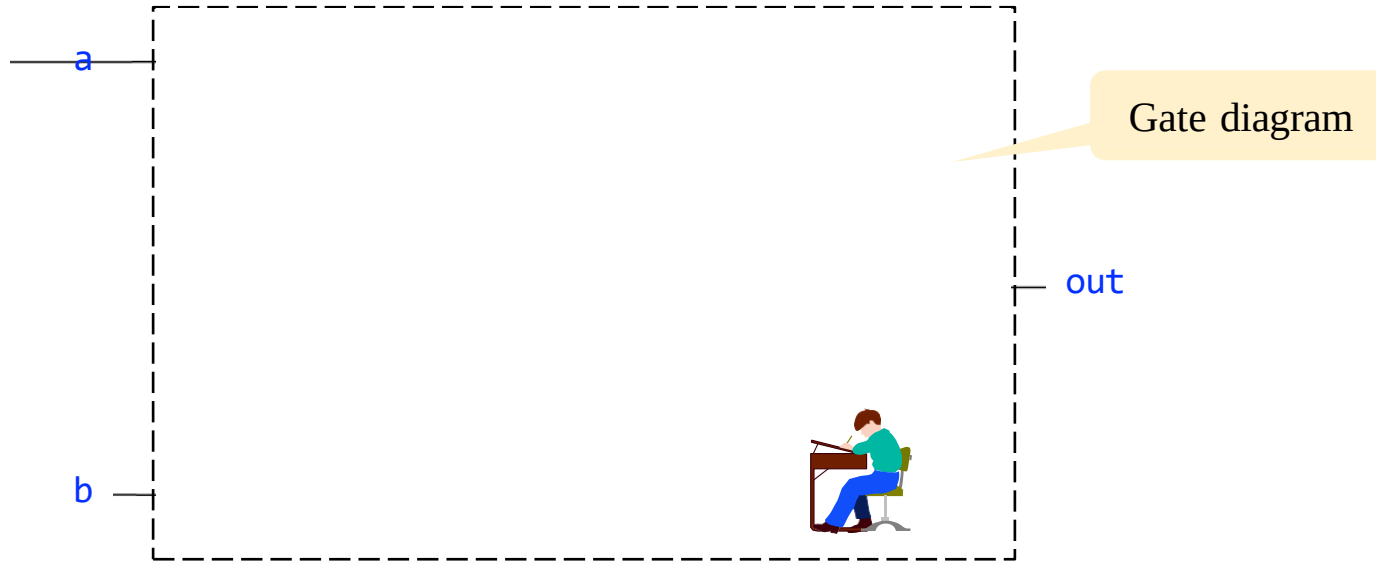
```
/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
    IN a, b;
    OUT out;
    PARTS:
        // Missing implementation
}
```

HDL stub file

```
/** Chips set (APIs): */
...
Not (in=, out= );
And (a=, b=, out= );
Or (a=, b=, out= );
Xor (a=, b=, out= );
...
```



Design: Implementation



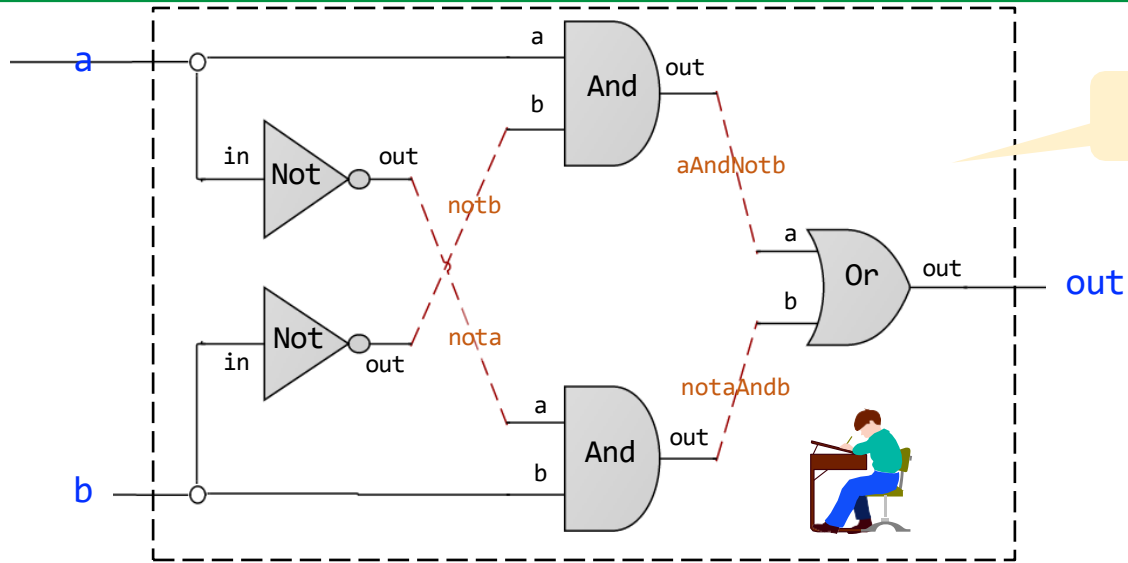
```
/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
  IN a, b;
  OUT out;
  PARTS:
    // Missing implementation
}
```

HDL stub file

```
/** Chips set (APIs): */
...
Not (in=, out= );
And (a=, b=, out= );
Or (a=, b=, out= );
Xor (a=, b=, out= );
...
```



Design: Implementation



Gate diagram

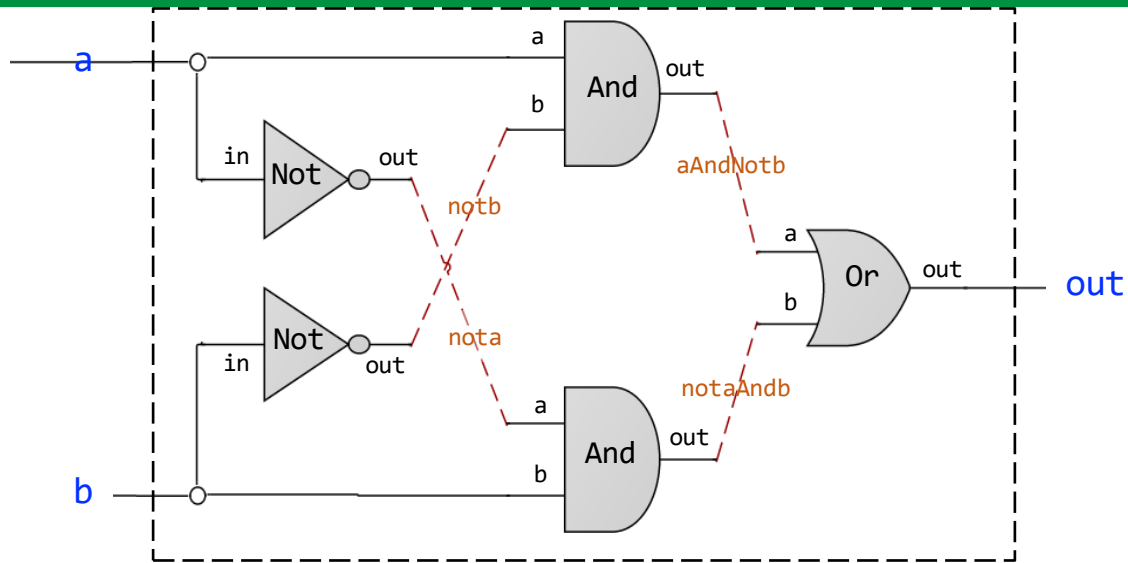
```
/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
  IN a, b;
  OUT out;
  PARTS:
    // Missing implementation
}
```

HDL stub file

```
/** Chips set (APIs): */
...
Not (in=, out= );
And (a=, b=, out= );
Or (a=, b=, out= );
Xor (a=, b=, out= );
...
```



Design: Implementation



```

/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
  IN a, b;
  OUT out;
  PARTS:
    // Missing implementation
}

```



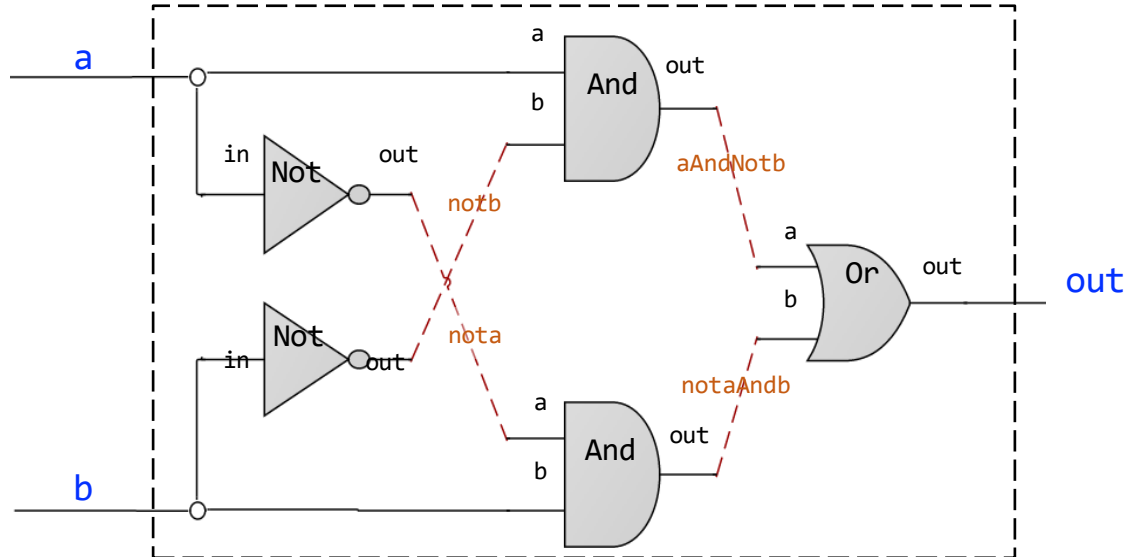
HDL stub file

```

/** Chips set (APIs): */
...
Not (in=, out= );
And (a=, b=, out= );
Or (a=, b=, out= );
Xor (a=, b=, out= );
...

```

Design: Implementation



```
/** out = (a And Not(b)) Or (Not(a) And b) */
```

```
CHIP Xor {
```

```
  IN a, b;
```

```
  OUT out;
```

```
  PARTS:
```

```
  Not (in=a, out=nota);
```

```
  Not (in=b, out=notb);
```

```
  And (a=a, b=notb, out=aAndNotb);
```

```
  And (a=nota, b=b, out=notaAndb);
```

```
}
```



```
/** Chips set (APIs): */
```

```
...
```

```
Not (in=, out= );
```

```
And (a=, b=, out= );
```

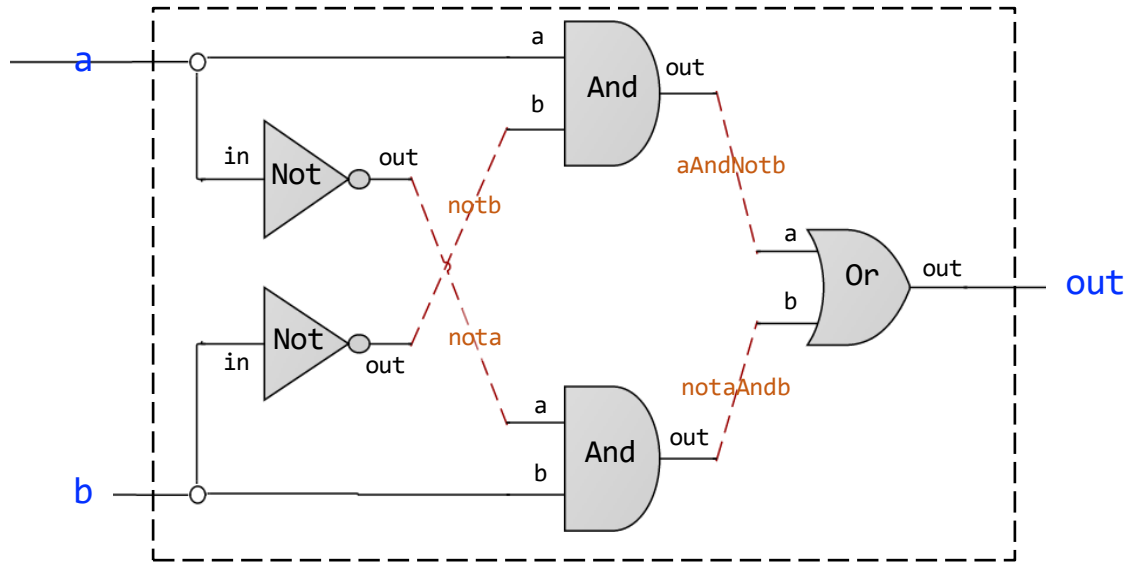
```
Or (a=, b=, out= );
```

```
Xor (a=, b=, out= );
```

```
...
```



Design: Implementation



```
/** out = (a And Not(b)) Or (Not(a) And b) */
```

```
CHIP Xor {
```

```
  IN a, b;
```

```
  OUT out;
```

```
  PARTS:
```

```
  Not (in=a, out=nota);
```

```
  Not (in=b, out=notb);
```

```
  And (a=a, b=notb, out=aAndNotb);
```

```
  And (a=nota, b=b, out=notaAndb);
```

```
}
```



```
/** Chips set (APIs): */
```

```
...
```

```
Not (in=, out= );
```

```
And (a=, b=, out= );
```

```
Or (a=, b=, out= );
```

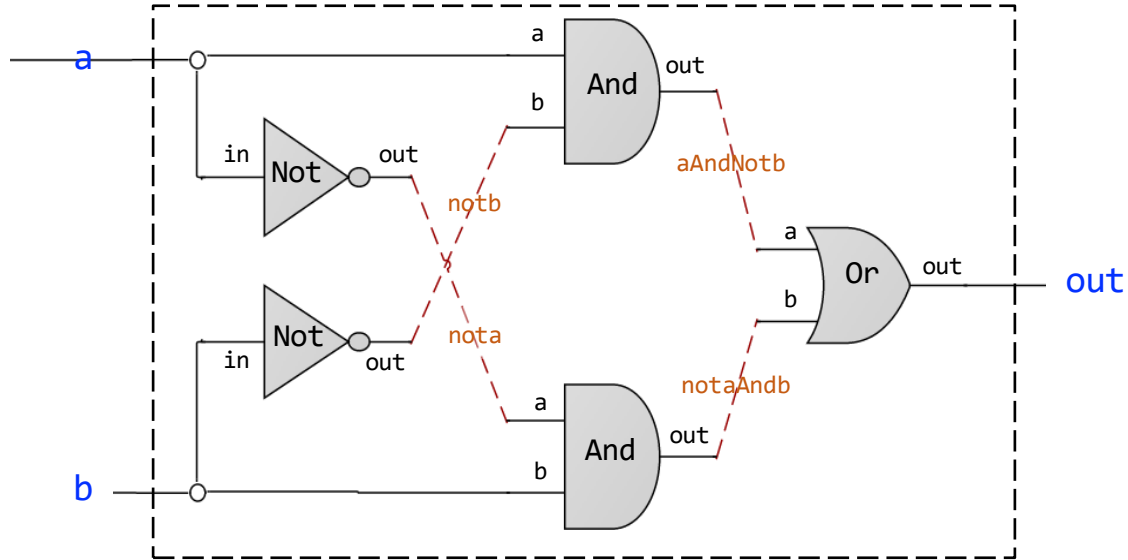
```
Xor (a=, b=, out= );
```

```
...
```

Question: What is the missing HDL line?



Design: Implementation



```

/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}

```



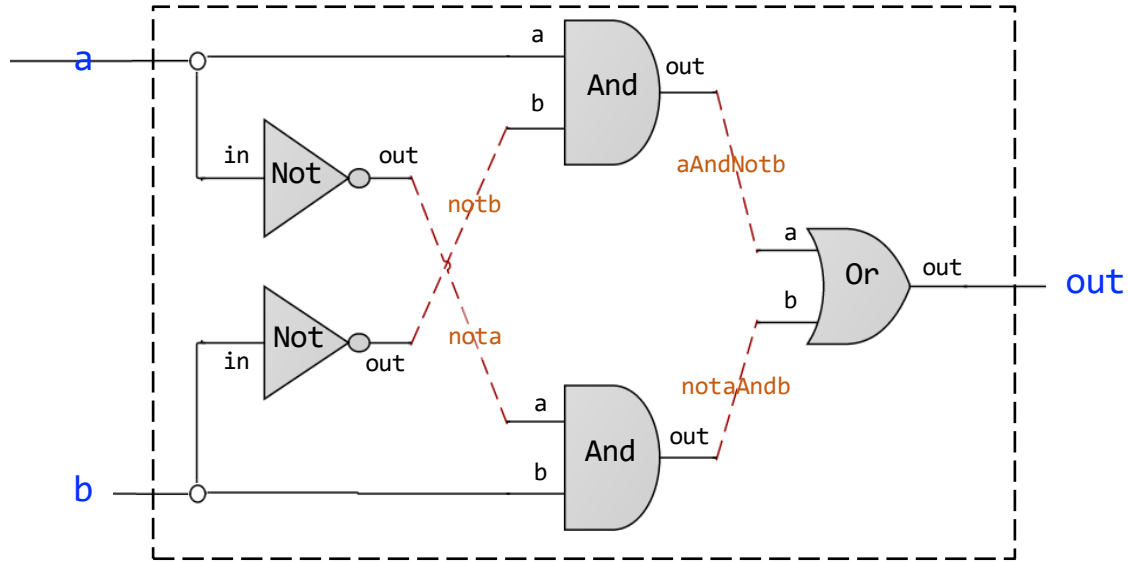
```

/** Chips set (APIs): */
...
Not (in=, out= );
And (a=, b=, out= );
Or (a=, b=, out= );
Xor (a=, b=, out= );
...

```



Interface / Implementation



gate
interface

gate
Implement-
-ation

```
/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
```

A logic gate has:

- One interface
- Many possible implementations



Hardware description languages

Observations

- HDL: a functional / declarative language
- An HDL program can be viewed as a textual / declarative specification of a chip diagram
- The order of HDL statements is insignificant.

```
/** out = (a And Not(b)) Or (Not(a) And b) */  
CHIP Xor {  
  IN a, b;  
  OUT out;  
  
  PARTS:  
    Not (in=a, out=nota);  
    Not (in=b, out=notb);  
    And (a=a, b=notb, out=aAndNotb);  
    And (a=nota, b=b, out=notaAndb);  
    Or (a=aAndNotb, b=notaAndb, out=out);  
}
```



Hardware description languages

Common HDLs

- VHDL
- Verilog
- ...

Our HDL

- Captures the essence of other HDLs
- Minimal and simple
- Provides all you need for this course
- Documentation:
[The Elements of Computing Systems / Appendix 2: HDL](#)

```

/** out = (a And Not(b)) Or (Not(a) And b) */
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}

```



Example: Building an **AND** gate



And.cmp

a	b	out
0	0	0
0	1	0
1	0	0
1	1	1

Contract:

When running your **.hdl** on our **.tst**, your **.out** should be the same as our **.cmp**.

And.hdl

```
CHIP And
{
    IN  a, b;
    OUT out;
    // implementation missing
}
```

And.tst

```
load And.hdl,
output-file And.out,
compare-to And.cmp,
output-list a b out;
```

```
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```



Building an **AND** gate



Interface: $\text{And}(a,b) = 1$ exactly when $a=b=1$



And.hdl

```
CHIP And
{
    IN  a, b;
    OUT out;
    // implementation missing
}
```



Building an **AND** gate



Implementation: $\text{And}(a,b) = \text{Not}(\text{Nand}(a,b))$



And.hdl

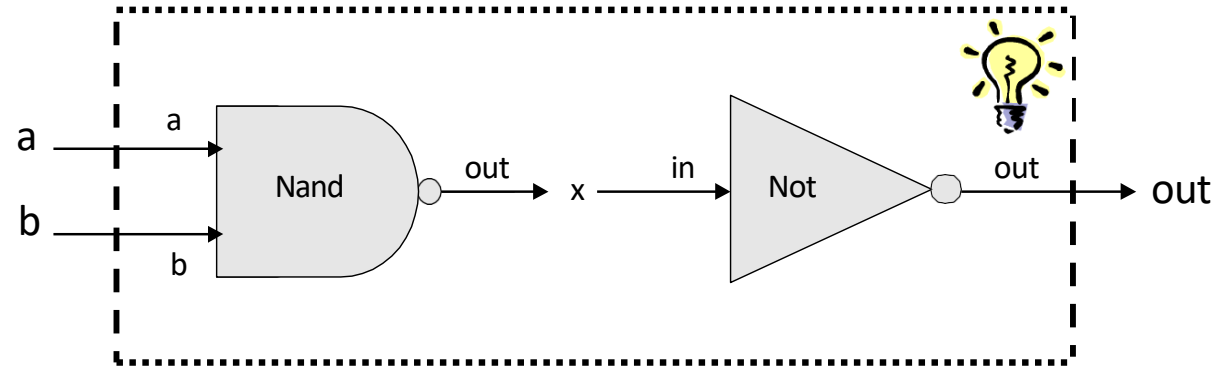
```
CHIP And
{
    IN  a, b;
    OUT out;
    // implementation missing
}
```



Building an **AND** gate



Implementation: $\text{And}(a,b) = \text{Not}(\text{Nand}(a,b))$



And.hdl

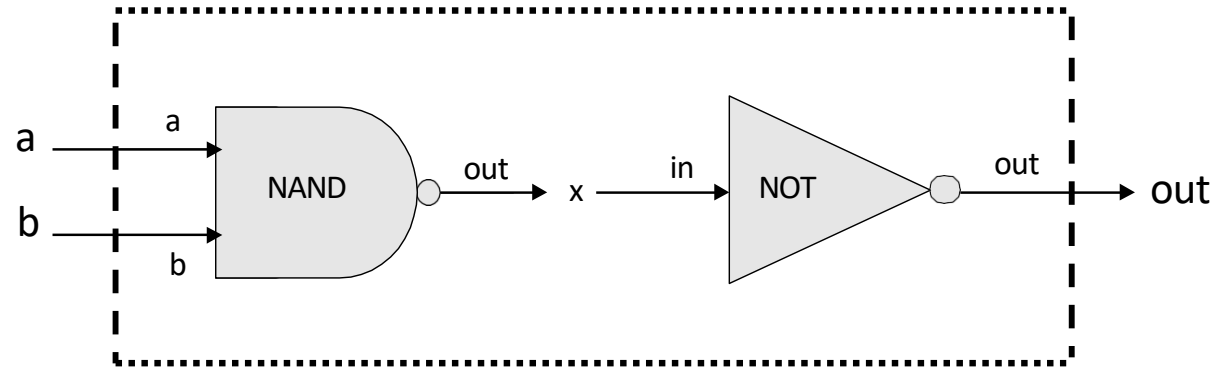
```
CHIP And
{
    IN  a, b;
    OUT out;
    // implementation missing
}
```



Building an **AND** gate



Implementation: $\text{And}(a,b) = \text{Not}(\text{Nand}(a,b))$



And.hdl

```

CHIP And
{
    IN  a, b;
    OUT out;
    Nand(a = a,
        b = b,
        out = x);
    Not(in = x, out = out)
}

```



Chapter 1: Boolean logic

Theory

- Basic concepts
- Nand

Practice

- ✓ Logic gates
- ✓ HDL
- ➔ Hardware simulation
 - Multi-bit buses

Project 1

- Introduction
- Chips
- Guidelines

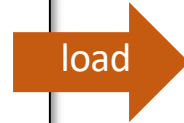
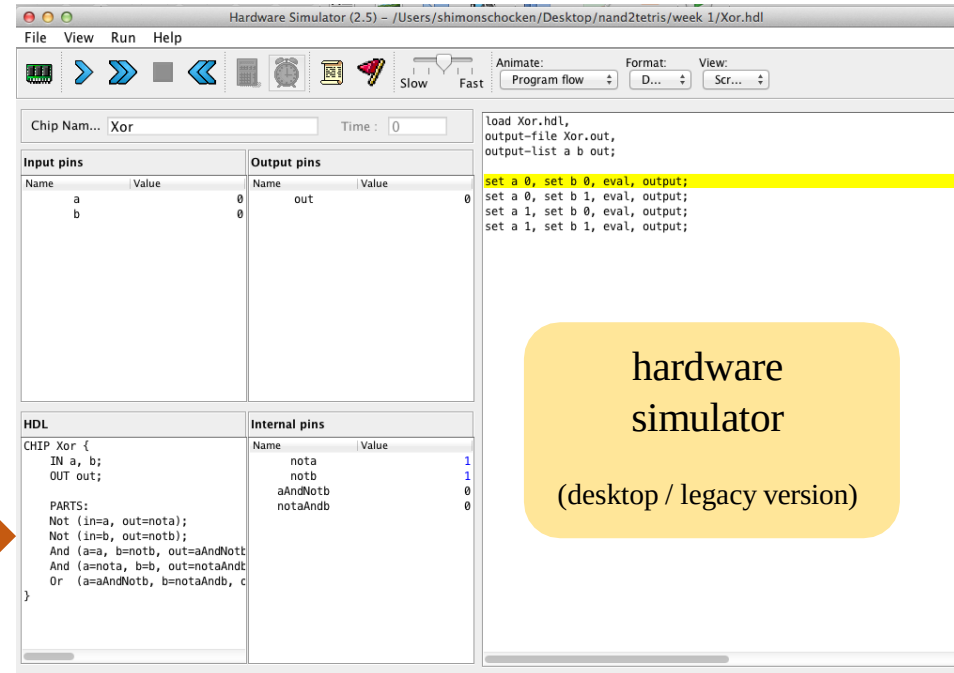


Hardware simulation (in a nutshell)

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
```

HDL code

hardware simulator
(desktop / legacy version)

Simulation options

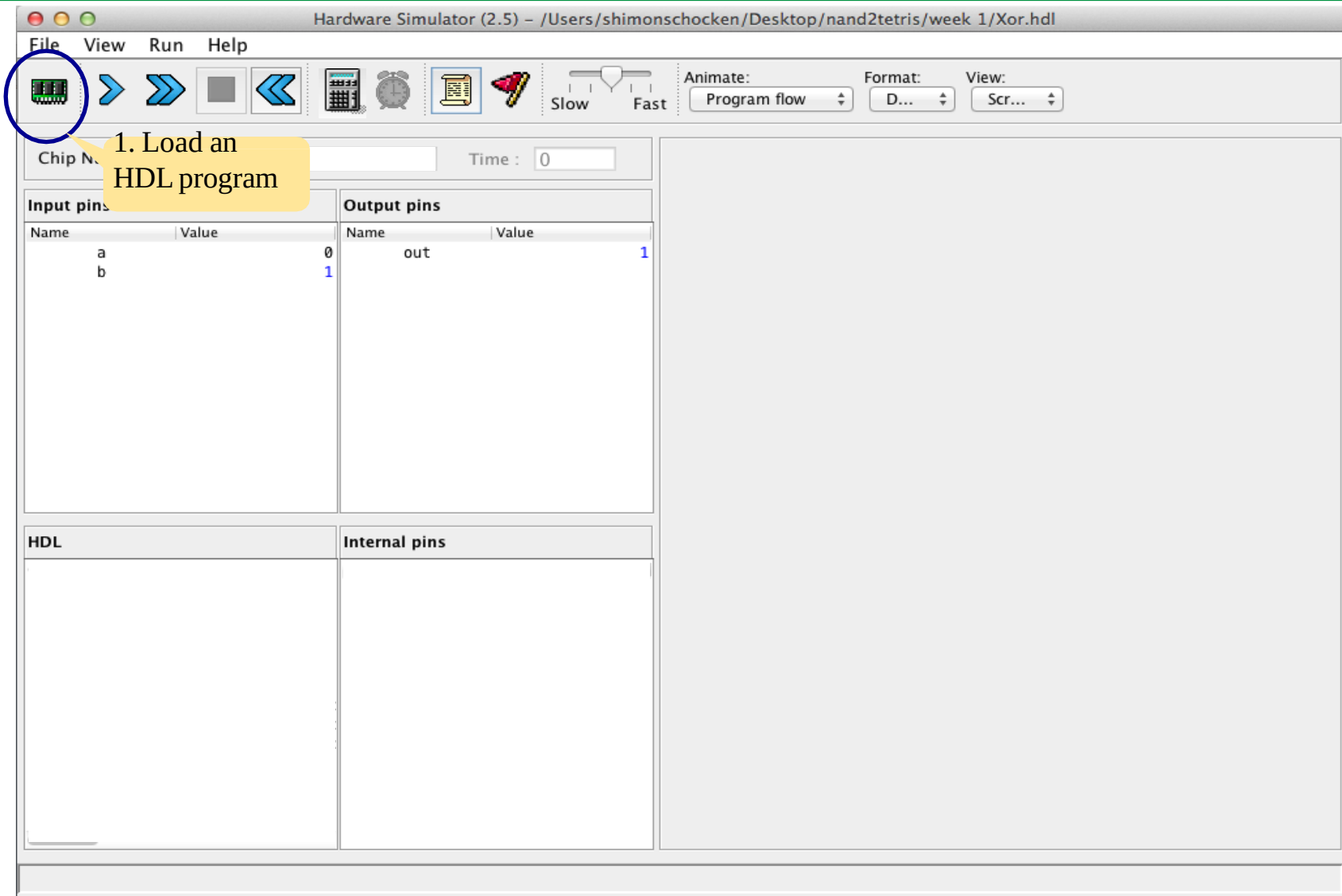


Interactive

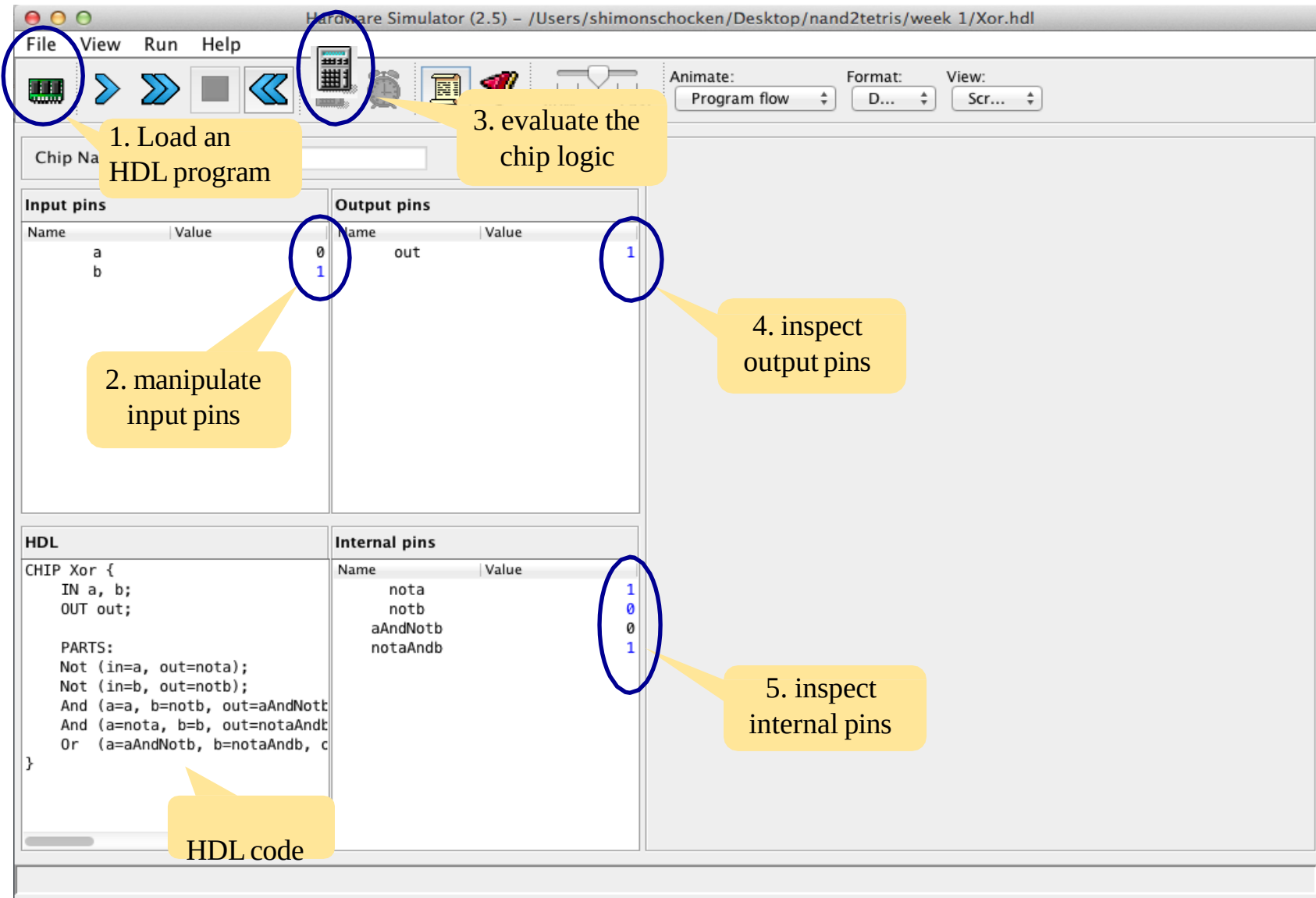
- Script-based.



Interactive simulation



Interactive simulation



Hardware Simulator (2.5) - /Users/shimonschocken/Desktop/nand2tetris/week 1/Xor.hdl

File View Run Help

1. Load an HDL program

3. evaluate the chip logic

2. manipulate input pins

4. inspect output pins

5. inspect internal pins

HDL code

Input pins		Output pins	
Name	Value	Name	Value
a	0	out	1
b	1		

Internal pins	
Name	Value
nota	1
notb	0
aAndNotb	0
notaAndb	1

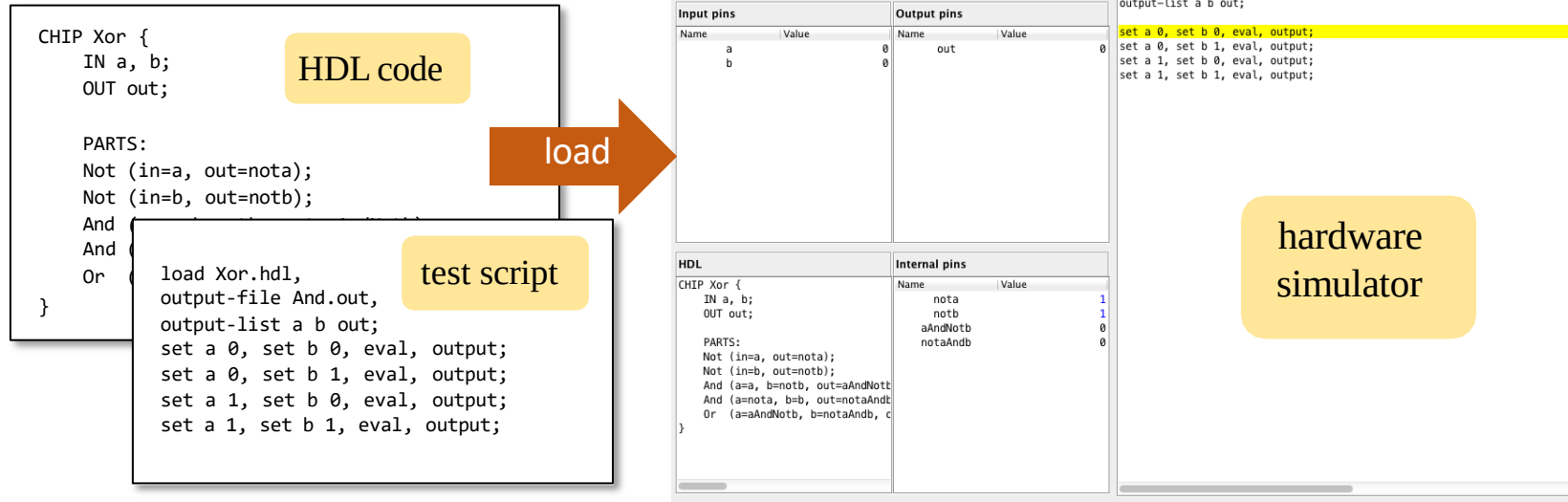
```

CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
  
```



Hardware simulation in a nutshell



Simulation options



Interactive



Script-based.



Script-based simulation

Xor.hdl

```
CHIP Xor {  
  IN a, b;  
  OUT out;  
  
  PARTS:  
    Not (in=a, out=nota);  
    Not (in=b, out=notb);  
    And (a=a, b=notb, out=aAndNotb);  
    And (a=nota, b=b, out=notaAndb);  
    Or (a=aAndNotb, b=notaAndb, out=out);  
}
```

Xor.tst

```
load Xor.hdl;  
set a 0, set b 0, eval;  
set a 0, set b 1, eval;  
set a 1, set b 0, eval;  
set a 1, set b 1, eval;
```

test script = sequence of
commands to the simulator

Benefits:

- “Automatic” testing
- Replicable testing.



Script-based simulation, with an output file

Xor.hdl

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
```

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

Xor.out

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

test
script

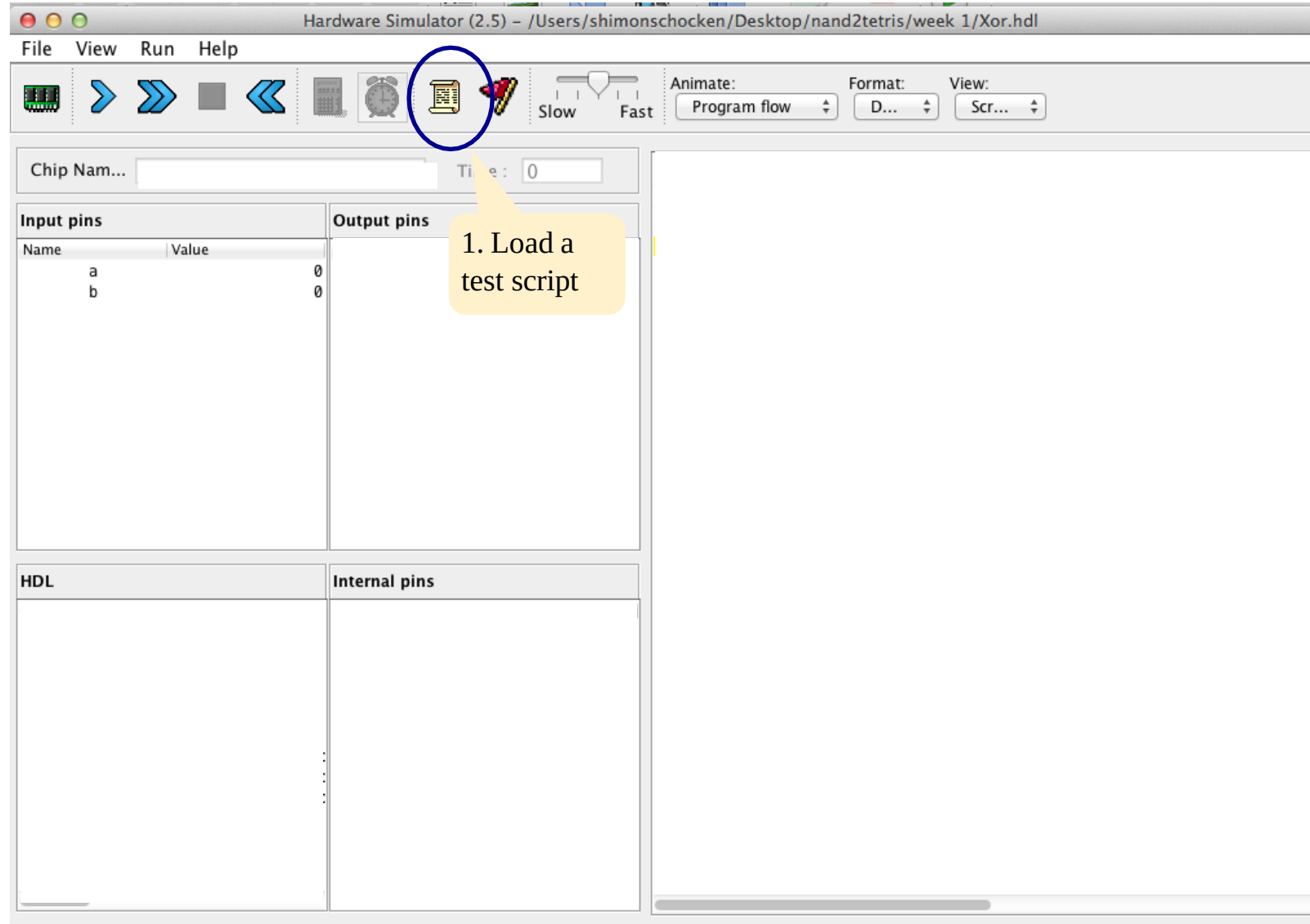
Output File, created
by the test script,
as a side-effect of the
simulation process

The logic of a typical test script

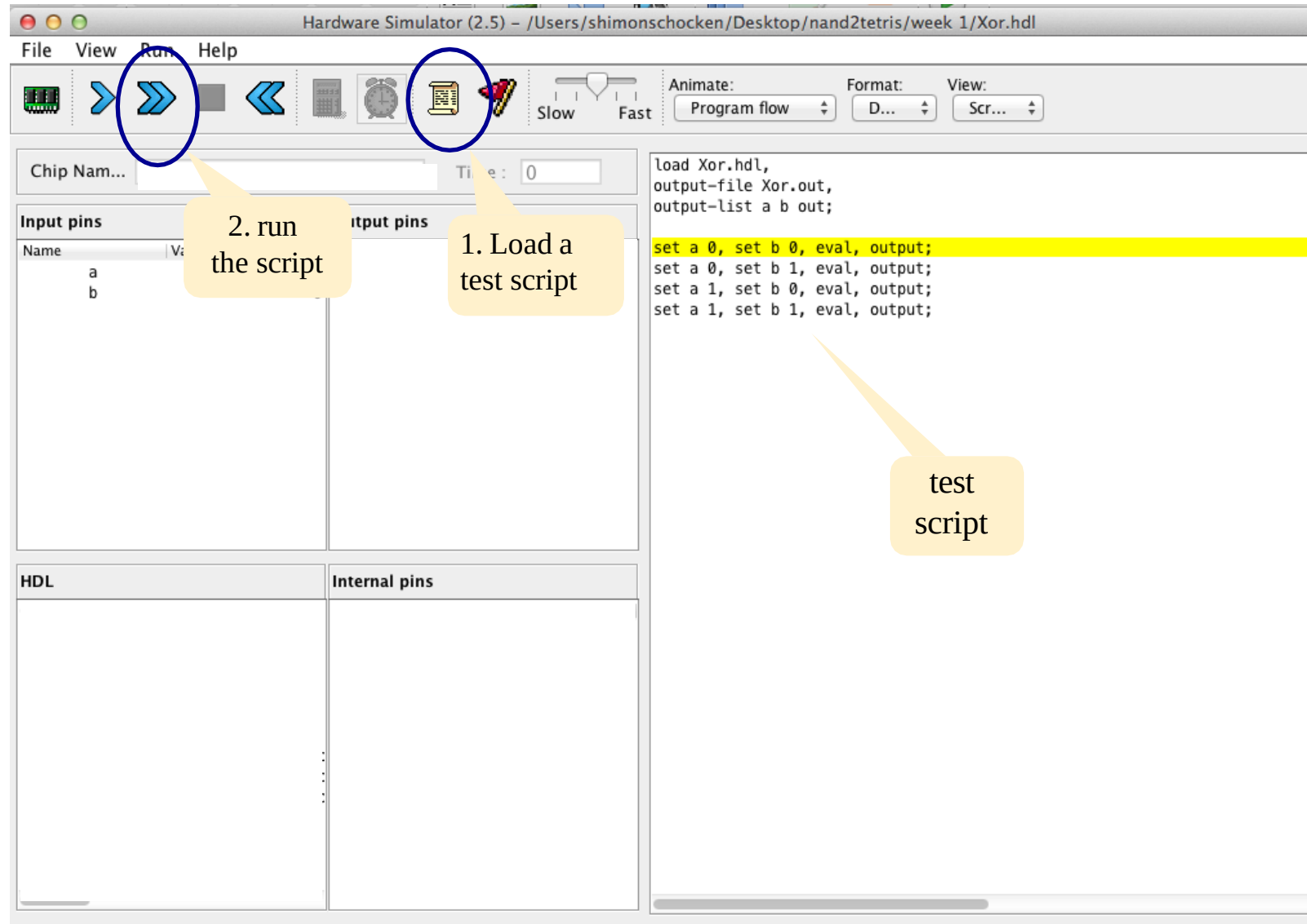
- Initialize:
 - ❑ Loads an HDL file
 - ❑ Creates an empty output file
 - ❑ Lists the names of the pins whose values will be written to the output file
- Repeat:
 - ❑ Set (inputs) , eval (chip logic) , output (print)



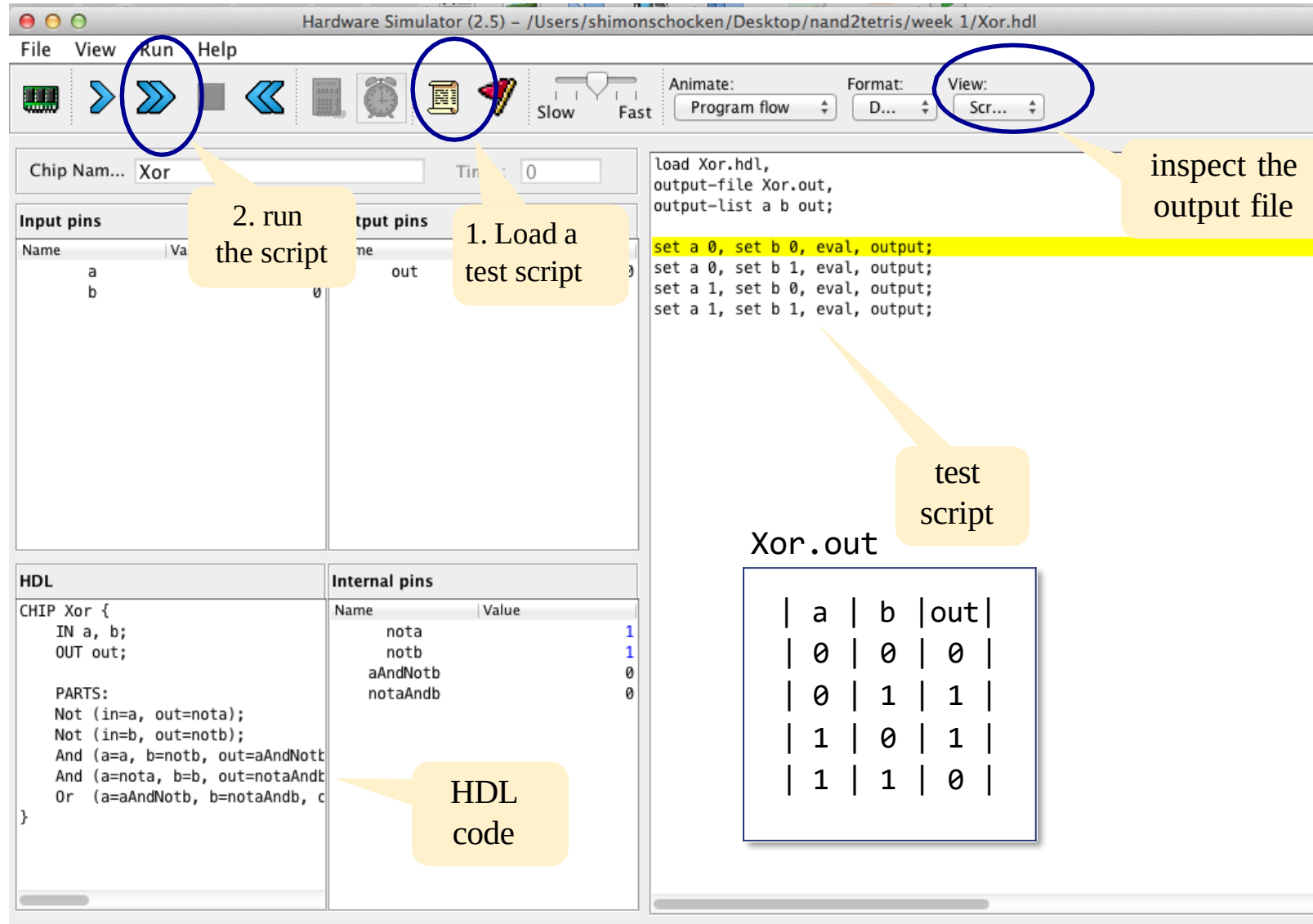
Script-based simulation



Script-based simulation



Script-based simulation



The screenshot shows the Hardware Simulator (2.5) interface. The title bar indicates the file path: `/Users/shimonschocken/Desktop/nand2tetris/week 1/Xor.hdl`. The menu bar includes File, View, Run, and Help. The toolbar contains various icons, with the 'Run' button (a blue double arrow) circled in blue. A yellow callout bubble points to it with the text '2. run the script'. Another yellow callout bubble points to the 'Load' icon (a document with a magnifying glass) with the text '1. Load a test script'. A third yellow callout bubble points to the 'View' dropdown menu, which is set to 'Scr...' (Script), with the text 'inspect the output file'. The main window is divided into several sections:

- Input pins:** A table with columns 'Name' and 'Value'. It shows inputs 'a' and 'b' with values '0' and '0' respectively.
- Output pins:** A table with columns 'Name' and 'Value'. It shows output 'out' with a value of '0'.
- HDL:** A text area containing the HDL code for the Xor chip. A yellow callout bubble points to it with the text 'HDL code'.
- Internal pins:** A table with columns 'Name' and 'Value'. It shows internal signals: 'nota' (1), 'notb' (1), 'aAndNotb' (0), and 'notaAndb' (0).
- Script:** A text area containing a test script. A yellow callout bubble points to it with the text 'test script'. The script includes:


```
load Xor.hdl,
output-file Xor.out,
output-list a b out;

set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```
- Xor.out:** A table showing the output results. A yellow callout bubble points to it with the text 'Xor.out'.



Script-based simulation

Xor.hdl

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
```

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

Xor.out

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0



Script-based simulation, with a compare file

Xor.hdl

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
```

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
compare-to Xor.cmp,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

Simulation-with-compare-file logic

- When each output command is executed, the outputted line is compared to the corresponding line in the compare file
- If the two lines are not the same, the simulator throws a comparison error.

Xor.out

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Xor.cmp

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

compare



Script-based simulation, with a compare file

Xor.hdl

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}
```

Xor.tst

```
load Xor.hdl,
output-file Xor.out,
compare-to Xor.cmp,
output-list a b out;
set a 0, set b 0, eval, output;
set a 0, set b 1, eval, output;
set a 1, set b 0, eval, output;
set a 1, set b 1, eval, output;
```

Demos:

[Experimenting with Built-In Chips](#)

[Building and Testing HDL-based Chips](#)

[Script-Based Chip Testing](#)

Xor.out

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Xor.cmp

a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

compare



Project materials: www.nand2tetris.org

From NAND to Tetris

Building a Modern Computer From First Principles



Home

Projects

Book

Software

Media

Cool Stuff

Terms

Q&A

About

Project 1: Logic Gates

← Project 1 web site

Background

A typical computer architecture is based on a set of elementary logic gates like And, Or, etc., as well as their bit-wise versions And16, Or16, etc. (in a 16-bit machine). This project engages you in the construction of a typical set of elementary gates. These gates form the elementary building blocks from which more complex chips will be later constructed.

Objective

Build all the logic gates described in Chapter 1 (see list below), yielding a basic chip-set. The only building blocks that you can use in this project are primitive Nand gates and the composite gates that you will gradually build on top of them.

Chips

Chip (HDL)	Function	Test Script	Compare File
Nand	Nand gate (primitive)		
Not	Not gate	Not.tst	Not.cmp
And	And gate	And.tst	And.cmp
Or	Or gate	Or.tst	Or.cmp
Xor	Xor gate	Xor.tst	Xor.cmp
Mux	Mux gate	Mux.tst	Mux.cmp
DMux	DMux gate	DMux.tst	DMux.cmp
Not16	16-bit Not	Not16.tst	Not16.cmp

← And.hdl, And.tst, And.cmp files



Chapter 1: Boolean logic

Theory

- Basic concepts
- Nand

Practice

- ✓ Logic gates
- ✓ HDL
- ✓ Hardware simulation
- ➔ Multi-bit buses

Project 1

- Introduction
- Chips
- Guidelines



Multi-bit bus

- Sometimes we wish to manipulate a *sequence of bits* as a single entity
- Such a multi-bit entity is called “bus”

Example: 16-bit bus

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	1	0	0	0	1	1	0	1	1	1	0	1

MSB = Most significant bit

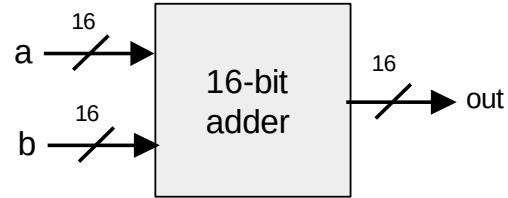
LSB = Least significant bit



Working with buses: Examples

```
/* Adds two 16-bit values. */
CHIP Adder {
  IN a[16], b[16];
  OUT out[16];

  PARTS:
  ...
}
```



	15	...	1	0
a:	1	...	1	1
b:	0	...	1	0
c:	0	...	0	1
out:	1	...	1	0

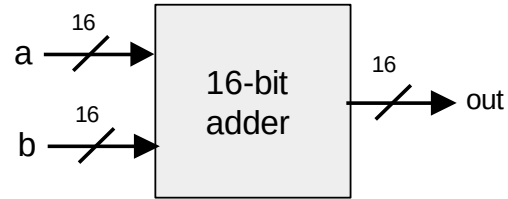
```
/* Adds three 16-bit inputs. */
CHIP Adder3Way {
  IN a[16], b[16], c[16];
  OUT out[16];
}
```



Working with buses: Examples

```
/* Adds two 16-bit values. */
CHIP Adder {
  IN a[16], b[16];
  OUT out[16];

  PARTS:
  ...
}
```



	15	...	1	0
a:	1	...	1	1
b:	0	...	1	0
c:	0	...	0	1
out:	1	...	1	0

```
/* Adds three 16-bit inputs. */
CHIP Adder3Way {
  IN a[16], b[16], c[16];
  OUT out[16];

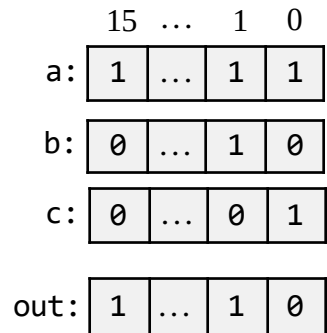
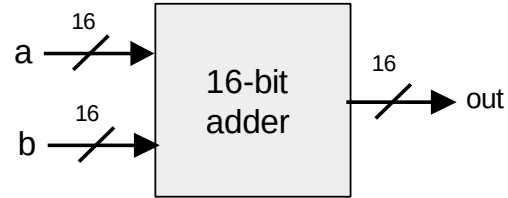
  PARTS:
  Adder(a= , b= , out= );
  Adder(a= , b= , out= );
}
```



Working with buses: Examples

```
/* Adds two 16-bit values. */
CHIP Adder {
  IN a[16], b[16];
  OUT out[16];

  PARTS:
  ...
}
```



```
/* Adds three 16-bit inputs. */
CHIP Adder3Way {
  IN a[16], b[16], c[16];
  OUT out[16];

  PARTS:
  Adder(a=a, b=b, out=ab);
  Adder(a=ab, b=c, out=out);
}
```

n-bit value (bus) can be treated as a single entity

Creates an internal bus pin (ab)



Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
   0 otherwise. */
```

```
CHIP And {
  IN a, b;
  OUT out;
  ...
}
```

a:

3	2	1	0
0	1	1	1

out:

0

```
/* 4-way And: Ands 4 bits. */
```

```
CHIP And4Way {
  IN a[4];
  OUT out;
```



Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
   0 otherwise. */
```

```
CHIP And {
  IN a, b;
  OUT out;
  ...
}
```

a:

3	2	1	0
0	1	1	1

out:

0

```
/* 4-way And: Ands 4 bits. */
```

```
CHIP And4Way {
  IN a[4];
  OUT out;
```

PARTS:

```
And(a=      , b=      , out=      );
And(a=      , b=      , out=      );
And(a=      , b=      , out=      );
```

```
}
```



Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
   0 otherwise. */
```

```
CHIP And {
  IN a, b;
  OUT out;
  ...
}
```

a:

3	2	1	0
0	1	1	1

out:

0

```
/* 4-way And: Ands 4 bits. */
```

```
CHIP And4Way {
  IN a[4];
  OUT out;
```

PARTS:

```
And(a=a[0],   b=a[1], out=and01);
And(a=and01,  b=a[2], out=and012);
And(a=and012, b=a[3], out=out);
```

```
}
```

Input bus pins can be subscripted.



Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
   0 otherwise. */
```

```
CHIP And {
  IN a, b;
  OUT out;
  ...
}
```

a:

3	2	1	0
0	1	1	1

out:

0

```
/* 4-way And: Ands 4 bits. */
```

```
CHIP And4Way {
  IN a[4];
  OUT out;
```

PARTS:

```
And(a=a[0], b=a[1], out=and01);
And(a=and01, b=a[2], out=and012);
And(a=and012, b=a[3], out=out);
```

```
}
```

Input bus pins can be subscripted.

```
/* Bit-wise And of two 4-bit inputs */
```

```
CHIP And4 {
  IN a[4], b[4];
  OUT out[4];
```

a:

3	2	1	0
0	1	0	1

b:

0	0	1	1
---	---	---	---

out:

0	0	0	1
---	---	---	---



Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
   0 otherwise. */
```

```
CHIP And {
  IN a, b;
  OUT out;
  ...
}
```

a:

3	2	1	0
0	1	1	1

out:

0

```
/* 4-way And: Ands 4 bits. */
```

```
CHIP And4Way {
  IN a[4];
  OUT out;
```

PARTS:

```
And(a=a[0], b=a[1], out=and01);
And(a=and01, b=a[2], out=and012);
And(a=and012, b=a[3], out=out);
```

```
}
```

Input bus pins can be subscripted.

a:

3	2	1	0
0	1	0	1

b:

0	0	1	1
---	---	---	---

out:

0	0	0	1
---	---	---	---

```
/* Bit-wise And of two 4-bit inputs */
```

```
CHIP And4 {
  IN a[4], b[4];
  OUT out[4];
```

PARTS:

```
And(a= , b= , out= );
And(a= , b= , out= );
And(a= , b= , out= );
And(a= , b= , out= );
```

```
}
```



Working with individual bits within buses

```
/* Returns 1 if a==1 and b==1,
   0 otherwise. */
```

```
CHIP And {
  IN a, b;
  OUT out;
  ...
}
```

a:

3	2	1	0
0	1	1	1

out:

0

```
/* 4-way And: Ands 4 bits. */
```

```
CHIP And4Way {
  IN a[4];
  OUT out;
```

PARTS:

```
And(a=a[0], b=a[1], out=and01);
And(a=and01, b=a[2], out=and012);
And(a=and012, b=a[3], out=out);
```

```
}
```

Input bus pins can be subscripted.

```
/* Bit-wise And of two 4-bit inputs */
```

```
CHIP And4 {
  IN a[4], b[4];
  OUT out[4];
```

PARTS:

```
And(a=a[0], b=b[0], out=out[0]);
And(a=a[1], b=b[1], out=out[1]);
And(a=a[2], b=b[2], out=out[2]);
And(a=a[3], b=b[3], out=out[3]);
```

```
}
```

Output bus pins can be subscripted

Additional essential tips:

[The Elements of Computing Systems / Appendix 2: HDL](#)



Chapter 1: Boolean logic



Theory

- Basic concepts
- Nand



Practice

- Logic gates
- HDL
- Hardware simulation
- Multi-bit buses

Project 1

- Introduction
- Chips
- Guidelines

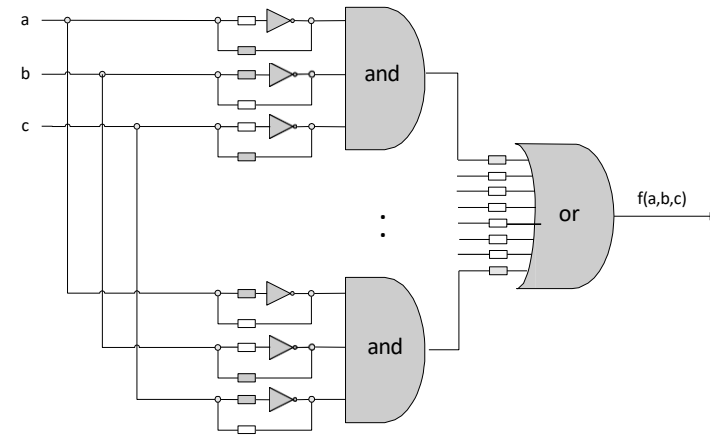


Project 1 tips

- Read the Introduction + Chapter 1 of the book
- Download the book's software suite
- Go through the hardware simulator tutorial
- Do Project 0 (optional)
- You're in business.



- Each Boolean function has a canonical representation
- The canonical representation is expressed in terms of And, Not, Or
- And, Not, Or can be expressed in terms of Nand alone
- Ergo, every Boolean function can be realized by a standard PLD consisting of Nand gates only
- Mass production
- Universal building blocks, unique topology
- Gates, neurons, atoms



End notes: Canonical representation

Whodunit story: Each suspect may or may not have an alibi (a), a motivation to commit the crime (m), and a relationship to the weapon found in the scene of the crime (w).
The police decides to focus attention only on suspects for whom the proposition **Not(a) And (m Or w)** is true.

Truth table of the "suspect" function $s(a, m, w) = \bar{a} + (m + w)$

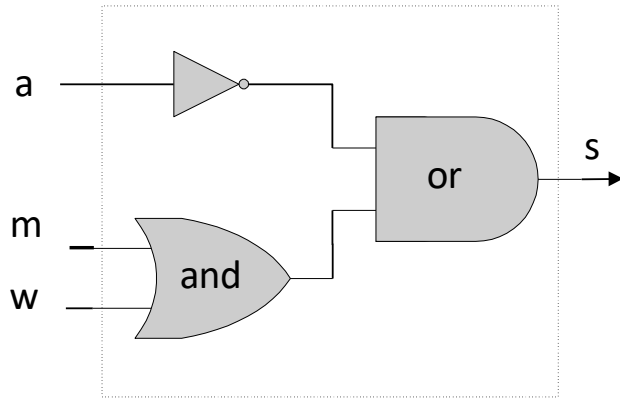
a	m	w	$minterm$	suspect(a,m,w)= not(a) and (m or w)
0	0	0	$m_0 = \bar{a} \bar{m} \bar{w}$	0
0	0	1	$m_1 = \bar{a} \bar{m} w$	1
0	1	0	$m_2 = \bar{a} m \bar{w}$	1
0	1	1	$m_3 = \bar{a} m w$	1
1	0	0	$m_4 = a \bar{m} \bar{w}$	0
1	0	1	$m_5 = a \bar{m} w$	0
1	1	0	$m_6 = a m \bar{w}$	0
1	1	1	$m_7 = a m w$	0

Canonical form: $s(a, m, w) = \bar{a} \bar{m} w + \bar{a} m \bar{w} + \bar{a} m w$

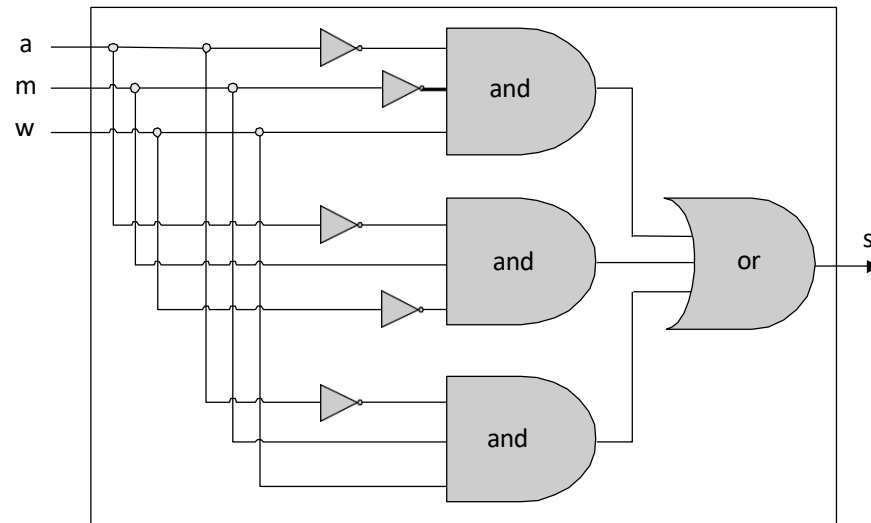


End notes: Canonical representation (cont.)

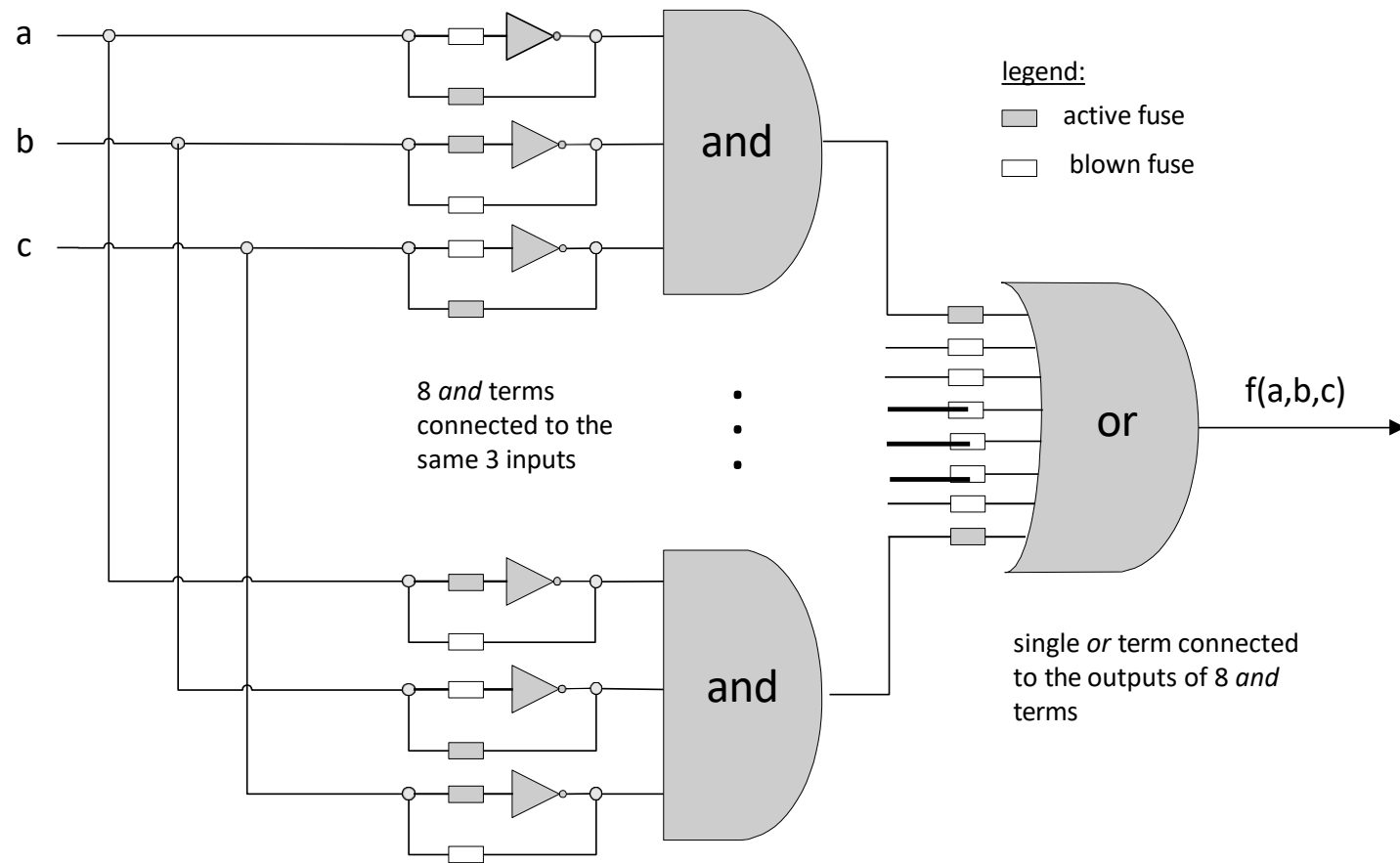
$$s(a, m, w) = \bar{a} \cdot (m + w)$$



$$s(a, m, w) = \bar{a}\bar{m}w + \bar{a}m\bar{w} + \bar{a}mw$$



End notes: Programmable Logic Device for 3-way functions



PLD implementation of $f(a,b,c) = a \bar{b} c + \bar{a} b c$

(the on/off states of the fuses determine which gates participate in the computation)



End notes: universal building blocks, unique topology

